



PDF Download
3676536.3676680.pdf
11 March 2026
Total Citations: 1
Total Downloads: 868

Latest updates: <https://dl.acm.org/doi/10.1145/3676536.3676680>

RESEARCH-ARTICLE

EasyPart: An Effective and Comprehensive Hypergraph Partitioner for FPGA-based Emulation

SHENGBO TONG, Tsinghua University, Beijing, China

HAOYUAN LI, Tsinghua University, Beijing, China

JIAHAO XU, Tsinghua University, Beijing, China

CHUNYAN PEI, Tsinghua University, Beijing, China

WENJIAN YU, Tsinghua University, Beijing, China

SHENGJUN LIU

[View all](#)

Open Access Support provided by:

[Tsinghua University](#)

Published: 09 April 2025

[Citation in BibTeX format](#)

ICCAD '24: 43rd IEEE/ACM International
Conference on Computer-Aided Design
October 27 - 31, 2024
NY, New York, USA

Conference Sponsors:
[SIGDA](#)

EasyPart: An Effective and Comprehensive Hypergraph Partitioner for FPGA-based Emulation

Shengbo Tong*, Haoyuan Li*, Jiahao Xu*, Chunyan Pei*, Wenjian Yu*, Shengjun Liu⁺, Jian Shen⁺

*Dept. Computer Science & Tech., BNRist, Tsinghua Univ., Beijing, China, ⁺HyperSilicon Inc., Wuxi, China

Abstract

Logic verification becomes more and more important for the design of large-scale digital integrated circuits (ICs). This makes FPGA-based hardware emulation an imperative step in the design flow, and how to effectively partition and map the circuit netlist into the multi-FPGA system (MFS) for emulation is of concern. In this paper, we present EasyPart, an effective and comprehensive hypergraph partitioner for the FPGA-based hardware emulation. EasyPart can handle the practical constraints in the MFS for logic emulation and includes novel techniques for pursuing minimum hop during topology-driven partitioning and treating the interconnection constraints. We have evaluated EasyPart against state-of-the-art partitioners on public benchmarks. The results show that EasyPart can reduce the cutsize with a comparable or shorter runtime. EasyPart is capable of finding non-hop solutions with better robustness and performance compared to previous work. It also achieves significant improvements in terms of time division multiplexing (TDM) ratio and maximum hop when tested on industrial cases.

Keywords

hypergraph partitioning, interconnection and topology constraints, multi-FPGA system, logic emulation

ACM Reference Format:

Shengbo Tong*, Haoyuan Li*, Jiahao Xu*, Chunyan Pei*, Wenjian Yu*, Shengjun Liu⁺, Jian Shen⁺. 2024. EasyPart: An Effective and Comprehensive Hypergraph Partitioner for FPGA-based Emulation. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD '24)*, October 27–31, 2024, New York, NY, USA. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3676536.3676680>

1 Introduction

Field-Programmable Gate Arrays (FPGAs) play a crucial role in the logic verification of digital integrated circuits (ICs), providing a flexible and efficient platform for validating and testing IC designs before fabrication. Because the capacity of a single FPGA is relatively limited, a large design usually needs to be partitioned into many subcircuits before it can be mapped into a multi-FPGA system (MFS) for logic emulation which largely accelerates the procedure of logic verification. The logic netlist of the design can be well described by a hypergraph with cell/gate denoted by graph vertex and interconnect

net denoted by hyperedge. Therefore, a hypergraph partitioner is imperative to the MFS for logic emulation.

However, computing the optimal partition for a hypergraph is a well-known NP-hard problem[11, 15], for which many heuristic algorithms have been proposed. hMETIS[12], along with other partitioners such as PaToH[7], employs a multilevel paradigm for hypergraph partitioning, which is a three-phase workflow composed of coarsening, initial partitioning, and refinement phases. KaHyPar[19] further improves the partition quality by utilizing a highly localized local search with flow-based techniques. HyperSF[2] and HyperEF[1] use spectral methods to replace the coarsening phase, achieving more efficiency without losing the key structural properties of the original hypergraph. SpecPart[5, 6] is a spectral-based partitioner that can produce embeddings of vertices and improve the quality of partition. These hypergraph partitioners focus on minimizing the cutsize, which depicts how many netlists are partitioned across different FPGAs. While for the FPGA-based emulation, the definition of cutsize should reflect more information regarding the actual connective structure among FPGAs, because it further influences the total delay and clock frequency, which is crucial for logic emulation.

Furthermore, the logic partitioning for MFS enforces more constraints that may not be considered in a typical hypergraph partitioner. These constraints include the multi-dimensional balance constraint, the fixed vertex constraint, the grouping constraint, the interconnect constraint, the topology constraint, etc. The recently proposed TritonPart[4] is an open-source partitioner that can handle multiple constraints, but it does not consider the interconnection constraint and the topology constraint. TopoPart[24] is the first work that focuses on the topology constraint, which pursues non-hop solutions by calculating the feasible FPGAs that each cell/gate can be assigned. Li et al.[16, 17] improved TopoPart and proposed a simple method to find the most beneficial moves during the refinement phase to alleviate the interconnection imbalance. However, all the existing methods are not practical or effective enough to handle all the MFS constraints for logic emulation.

Table 1: Comparison of different hypergraph partitioners.

	hMETIS[12]	PaToH[7]	KaHyPar[19]	TritonPart[4]	Li et al.[16, 17]	EasyPart (ours)
Constraints for FPGA-based emulation						
Multi-dimensional balance constraint	✓	✓	✓	✓	✓	✓
Fixed vertex constraint	✓	✓	✓	✓	✓	✓
Grouping constraint				✓		✓
Interconnection constraint					✓	✓
Topology constraint					✓	✓



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivs International 4.0 License.

ICCAD '24, October 27–31, 2024, New York, NY, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1077-3/24/10

<https://doi.org/10.1145/3676536.3676680>

In Table 1, we list the capabilities of main hypergraph partitioners regarding the treatment of practical constraints in FPGA-based emulation. From it, we can see that most of the existing works do not consider the grouping and interconnection/topology constraints of the MFS. In this work, we present an effective and comprehensive hypergraph partitioner, named EasyPart, which considers all these constraints in the FPGA-based emulation. Our main contributions are as follows.

- We present an effective hypergraph partitioning engine based on a practical objective function of cutsizes for logic partitioning[3], which reflects the direction of signal flow between FPGAs. Experiments show that our partitioning engine can reduce the cutsizes compared to the state-of-the-art partitioners (hMETIS[12], TritonPart[4]) with comparable or better runtime, on the Titan23 benchmark suite[18].
- We propose an enhanced topology-driven initial partitioning algorithm that can find non-hop solutions under the topology constraint, and minimize the maximum hop of routing paths between FPGAs if no feasible solution exists. It achieves the same or better performance than the best existing method[17] with more robustness.
- We adapt the whole multi-level partitioning process by keeping and considering the direction of signal flow between FPGAs to handle the interconnection/topology constraint. A comprehensive gain function to evaluate the benefit of vertex moves in the refinement phase is proposed. The experiments on industrial cases demonstrate that with the proposed approaches, the time division multiplexing (TDM) ratio and the maximum hop of routing paths can be significantly reduced.
- With the above contributions, we have developed an effective and comprehensive hypergraph partitioning framework, named **EasyPart**, considering the practical constraints for large-scale logic emulation with MFS.

2 Preliminaries

2.1 Problem Formulation of Hypergraph Partitioning

Table 2 provides a list of important terminologies and notations used throughout this paper. In a hypergraph $G(V, E)$, each hyperedge can connect more than two vertices. The basic objective of the hypergraph partitioning is to divide the vertices into K blocks $V_1, V_2, \dots, V_K$, such that the sum of weights of the hyperedges that connect vertices from different blocks is minimized, while satisfying the balance constraint:

$$\left(\frac{1}{K} - \epsilon\right) \sum_{v \in V} w(v) \leq \sum_{v \in V_i} w(v) \leq \left(\frac{1}{K} + \epsilon\right) \sum_{v \in V} w(v), \forall 1 \leq i \leq K \quad (1)$$

Here $w(v)$ denotes the weight of vertex v , and ϵ is a user-defined tolerance depicting in what degree the hypergraph is not partitioned into K equal-size parts[12]. Although most existing hypergraph partitioners consider the constraint (1), KaHyPar[16] only considers the upper bound in the inequality. Notice that a simple assumption is that the weight of every hyperedge is 1, and the optimization objective of the above problem is called *cut*. The problem is sometimes referred to as a min-cut optimization problem.

Table 2: Terminology and Notation

Variable	Definition
$G(V, E)$	Hypergraph for circuit netlist, where V denotes vertex set and E denotes hyperedge set
v_i	A vertex in hypergraph, representing a cell in circuit
e_i	A hyperedge in hypergraph, representing a net in circuit
$w(v)$	Weight of vertex v , i.e. the needed resource for the corresponding cell
$w(e)$	Weight of hyperedge e , i.e. the number of communicating signals on the corresponding net
$part(v)$	The block (FPGA) which vertex v is assigned to
$dist(v_i, v_j)$	Minimum distance between vertices v_i and v_j
$S(e)$	Set of the FPGA which the source node of the net corresponding to hyperedge e is assigned to
$D(e)$	Set of the FPGAs which the drain nodes of the net corresponding to hyperedge e are assigned to
$c(e)$	Cutsizes of hyperedge e
$\hat{G}(\hat{V}, \hat{E})$	Graph of MFS, where vertex set $\hat{V}$ denotes FPGAs and edge set $\hat{E}$ denotes the connectivity among FPGAs
$\hat{v}_i$	Vertex i in the MFS graph, denoting the i -th FPGA
$\hat{e}_i(\hat{v}_i, \hat{v}_j)$	An edge between vertices $\hat{v}_i$ and $\hat{v}_j$, representing the physical connection between two FPGAs
$cddt(v_i)$	Set of candidate FPGAs that cell v_i can be assigned to
$C(\hat{v}_i, \hat{v}_j)$	Cutsizes between two FPGAs $\hat{v}_i$ and $\hat{v}_j$
$IO(\hat{v}_i, \hat{v}_j)$	Supplied interconnection resources between $\hat{v}_i$ and $\hat{v}_j$
K	The number of blocks (FPGAs) for partition
ϵ	Imbalance factor
β	Threshold of $ cddt(v_i) $ for each cluster vertex
τ	Penalty factor on interconnection constraint

2.2 Considerations in FPGA-based Emulation

For FPGA-based emulation, there should be more considerations of constraints than the above basic hypergraph partitioning problem. Therefore, the *cut* in the last subsection is very simplified and more meaningful cutsizes definitions are needed.

2.2.1 Cutsizes Definitions. Differently from the simplest cutsizes definition *cut* (equals the sum of weights of the cut hyperedges), another definition multiplies the weight of each cut hyperedge by the number of its connected partitions before adding them up, which leads to the cutsizes metric named sum-of-external-degrees (*soed*) in [22]. While considering the logic partition problem, one should be aware that each hyperedge corresponds to a signal net which has a unique source node and maybe multiple drain nodes. Thus, neither the original *cut* nor *soed* reflects the nature of a signal flow. In this work, we use the following definition of cutsizes for cut hyperedge e :

$$c(e) = w(e) \cdot |D(e) \setminus S(e)|, \quad (2)$$

where $D(e)$ and $S(e)$ denote the sets of FPGAs containing the drain nodes and the source node corresponding to hyperedge e , respectively. “ $\setminus$ ” denotes the subtraction of set, and “ $|\cdot|$ ” denotes the cardinality of set. The objective function in the hypergraph partitioning problem for FPGA-based emulation is the sum of cutsizes for cut hyperedges, which also equals to the *connectivity* metric[9] or the $(K - 1)$ metric[3].

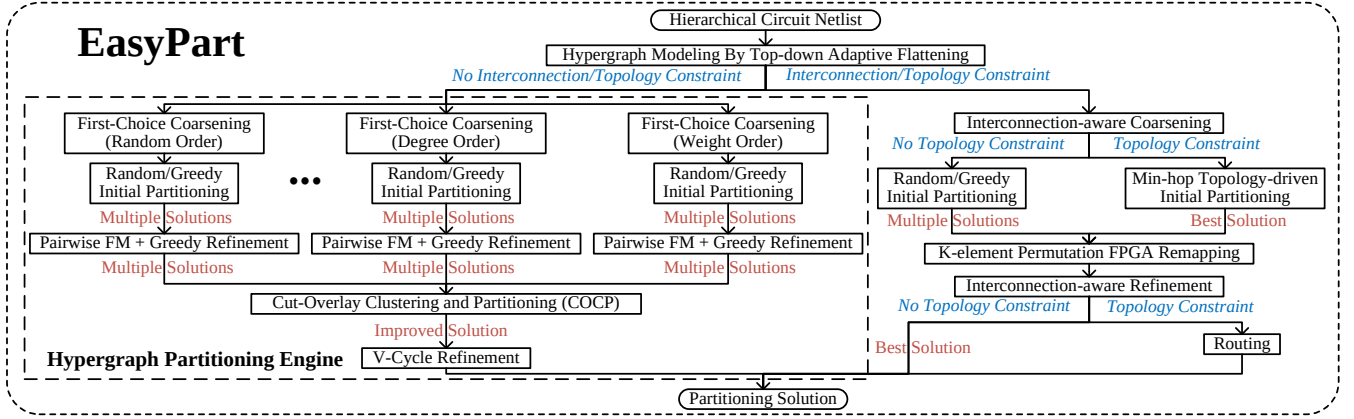


Figure 1: Flowchart of the EasyPart framework for the K-way partitioning problem in FPGA-based emulation.

Furthermore, for the K -way partitioning problem for MFS, the cutsizes between two blocks (FPGAs) can be defined:

$$C(\hat{v}_i, \hat{v}_j) = \sum_{\hat{v}_i \in S(e), \hat{v}_j \in D(e) \setminus S(e)} w(e) + \sum_{\hat{v}_j \in S(e), \hat{v}_i \in D(e) \setminus S(e)} w(e) \quad (3)$$

With it, one obtains a $K \times K$ cut matrix C . It is easy to prove:

$$\sum_{i=1}^K \sum_{i < j} C(\hat{v}_i, \hat{v}_j) = \sum_{e \in E} c(e) = soed - cut \quad (4)$$

where cut denotes the simplest cutsizes definition in the last subsection. Notice that (4) is the cutsizes metric considered in this work, which is relatively close to the $soed$ metric.

2.2.2 Multiple Constraints. As shown in Table 1, some partitioners only consider the single-dimensional balance constraint. This is not adequate for the problem of FPGA-based emulation, because there are multiple types of resources in an FPGA. So, the multi-dimensional balance constraint should be considered. This is realized by simply changing $w(v)$ in (1) to a vector $w(v) = (w_1, w_2, \dots, w_m)$, where m is the number of resource types. Besides, in some settings, certain circuit cell/gate must be assigned to a specific FPGA for debugging or other reasons. This is called fixed vertex constraint in hypergraph partitioning problem, and can be easily handled by forcing the partition result of this kind of vertices to their specific FPGAs and ignoring their placement in the partitioning process. There is another constraint called grouping constraint, as some cells/gates in the circuit are closely related to each other, and it is recommended or even forced to assign them to the same FPGA. To meet this requirement, one can merge the related vertices into a super vertex and treat it as a single vertex in the partitioning process.

For the practical MFS for logic emulation, there are more complex constraints regarding the interconnection among FPGAs, e.g., limited interconnection wires exist among FPGAs. So, if the cutsizes between two FPGAs exceeds the number of connection wires, TDM has to be used to resolve the congestion but leads to a larger signal delay. Moreover, some FPGA pairs may not have any wire connection in between (a special interconnect constraint). Simply ignoring this fact will lead to unavoidable routing after partitioning, which often results in longer signal paths with hops on intermediate FPGAs. These two

constraints are referred to as the interconnection constraint and the topology constraint respectively.

2.3 Interconnection-aware and Topology-driven Partitioning

If the partitioning algorithm does not consider the interconnection constraint among different FPGA pairs, it often causes a higher TDM ratio between some FPGAs[20], and thus larger maximum signal delay. To alleviate this problem, [16] defines a score function to reflect the imbalance degree of TDM usage and try to reduce the maximum TDM ratio during the refinement phase. Its drawback is that it overlooks the potential opportunities in other phases and does not fully exploit the advantages of the multilevel paradigm.

TopoPart[24] is the first work of hypergraph partitioning that focuses on topology constraint, aiming to find a non-hop solution. It solves the problem with a necessary and sufficient condition for feasible solutions and calculates the candidate set that contains all the available blocks for each vertex. [17] improves TopoPart by introducing a more effective candidate set updating method. However, both algorithms ignore the importance of the relative size of vertex degree in MFS when choosing the best block, and do not handle the partitioning problem well when there is no feasible solution.

3 Proposed Method

In this section, we present the proposed EasyPart framework for the K-way partitioning problem in FPGA-based emulation, whose flowchart is shown in Fig. 1. EasyPart includes an effective hypergraph partitioning engine that can achieve high-quality partitioning when there is no interconnection or topology constraint (Section 3.1). When the interconnection/topology constraint is considered, EasyPart can handle it by making adaptations to the whole partitioning process (Section 3.2). Moreover, if the topology constraint exists, EasyPart utilizes a novel topology-driven initial partitioning algorithm to find a solution with minimum hop (Section 3.3). Notice that some techniques in the hypergraph partitioning engine are also applied if possible, when EasyPart handles the situation with interconnection/topology constraint.

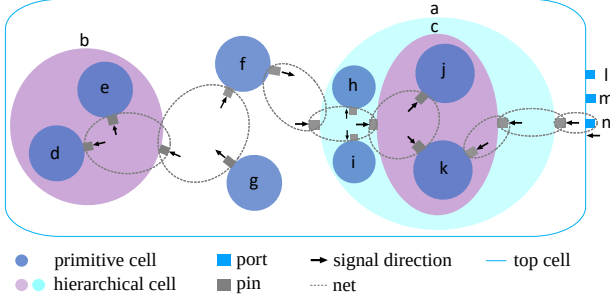
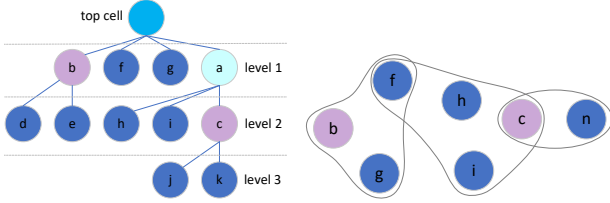


Figure 2: An abstract description of a part of hierarchical design.



(a) Hierarchy tree of cells (b) Result of hypergraph modeling

Figure 3: Different models for the hierarchical design in Fig. 2.

3.1 An Effective Hypergraph Partitioning Engine

EasyPart reads in a hierarchical circuit netlist. Before performing the hypergraph partitioning, a step of hypergraph modeling using a top-down adaptive flattening approach is developed (as shown in Fig. 1). It identifies when to flatten based on the resource ratio and net density of each cell. The hierarchical design can be modeled as a hierarchical design and Fig. 2 shows an abstract description of a part of the hierarchical design and Fig. 3(a) shows the corresponding hierarchical tree of cells. After adaptive flattening, the partially flattened design is transformed into a hypergraph to depict the cell-net relationship. Take the hierarchical design in Fig. 2 as an example. If during the adaptive flattening, cell *a* is flattened by one level and cells *b* and *c* are not flattened, the result of hypergraph modeling leads to the hypergraph shown in Fig. 3(b). The top-down adaptive flattening approach preserves some hierarchical structures from human designers to benefit the final quality of logic partition, while avoiding a long time for hypergraph modeling and the largest hypergraph from complete flattening fed into the subsequent partitioner. This is also helpful for debugging and analysis of the design.

Our partitioning engine consists of the following five steps.

3.1.1 Coarsening. In the coarsening phase, we use the first-choice (FC) scheme[14], which merges vertices *u* and *v* with higher connectivity, measured by the *heavy-edge* function:

$$r(u, v) = \sum_{u, v \in e} \frac{w_e}{|e| - 1}. \quad (5)$$

Large hyperedges are skipped to speed up the calculation of the heavy-edge functions. During the clustering process, we do not restrict the number of vertices to be merged into a super node unless their resource requirements reach a certain limit. We allow different vertex traversing orders, including random order, degree order, and weight order, to produce various partitioning results. If there are ties, we break them by favoring uncoarsened vertices. The time to stop

each round of coarsening is determined by a given coarsening ratio factor. After merging the vertices, we merge parallel hyperedges and remove single-vertex hyperedges by using a fast hash function in [19] to reduce $|E|$ and the number of (u, v) pairs in (5), thus reducing the complexity of the algorithm. Because the value of cutsize is not influenced by the connection of vertices within each hyperedge, we do not care about the order of vertices during the merging process if interconnection/topology constraint is not considered.

3.1.2 Initial Partitioning. When the balance constraint only considers the upper bound, we use the greedy growing partitioning[13] to get a relatively good initial solution. If no fixed vertices are given, we start from the one with the smallest degree and use shortest path algorithms to find the furthestmost vertices until each FPGA has at least one assigned node. For the balance constraint with both upper and lower bounds, we simply use random partitioning to assign each FPGA with one node alternately. After the initial partitioning, we use techniques in the following refinement phase to reduce the cutsize as much as possible. Keeping only one solution may lead to fluctuating results, and the best initial result does not guarantee the best final result. So, we keep multiple solutions and pass them all to the refinement phase like hMETIS[12].

3.1.3 Refinement. After different initial solutions are given, we refine them in parallel. At each level, we use different refinement methods to improve the partition quality, and partitions with cutsize 10% worse than the best solution are removed to save time[12]. The most common refinement method is the Fiduccia-Mattheyses (FM) algorithm[10], which is a local search algorithm that iteratively moves vertices between partitions to reduce cutsize. Here we choose the K-way pairwise FM (PM) algorithm to achieve better performance[8]. After that, we run iterations of greedy K-way refinement[14], which produce high-quality partitions while consuming limited time. Both methods use the definition of cutsize (4) to evaluate the benefit of moves. Because calculating gains of vertex moves and updating priority queues are closely related to $|E|$ and $|e|$, we also skip large hyperedges just as in the coarsening phase. Unlike the previous methods that only save the best partition result at the last level, we keep all the remained solutions and pass them to the next step.

3.1.4 Cut-Overlay Clustering and Partitioning (COCAP). After running the multilevel partitioning several times and generating a series of candidate solutions, we apply the COCAP, a technique to further improve the solution by combining information from all candidates[6].

3.1.5 V-Cycle Refinement. Lastly, we apply the V-cycle refinement to further improve the prior solution[12]. The multilevel partitioning will be performed several times. In the coarsening phase, the prior solution is used as a community constraint. Then the initial partitioning phase generates multiple solutions, and the prior solution is also kept as an initial solution. Finally, the refinement phase returns the best partitioning result.

3.2 Treatment of Interconnection/Topology Constraint

To handle the interconnection/topology constraint based on the engine above, we should make adaptations to all three main phases of the partitioning process, as follows.

3.2.1 Coarsening. Because now we have to consider the exact cutsizes between FPGAs instead of just the sum, we can only merge hyperedges with the same source node and drain nodes. However, the original merging process is unaware of the vertex order in each hyperedge. To solve this problem, we keep the source node and sort the drain nodes in each net, before using the hash function to merge them in an order-sensitive way.

3.2.2 Initial Partitioning. Instead of judging whether to keep an initial solution or not just based on the cutsizes, we now change the criterion to the TDM ratio, except when the topology constraint exists. If the topology constraint exists, we use the initial partitioning method described in Section 3.3. We also try to remap the FPGAs to get a better result when the interconnection resources between the FPGAs are not unified, because the partition result of the initial partitioning may not be optimized under the TDM ratio and routing cost. Therefore, we search for a K-element permutation P to reduce them, and then reassign each node v from $part(v)$ to $P(part(v))$. During the process, the total cutsizes remains unchanged. We use different heuristic methods to search for a better solution, including pairwise exchange and simulated annealing.

3.2.3 Refinement. This is the most important phase in handling the interconnection constraint, and we present an in-depth treatment to improve partitioning quality. First, we should restore all the large hyperedges since they now may lead to the use of unnecessary TDM in some corner cases. Because we use the PM algorithm which starts from the FPGA pairs that have the largest cutsizes in between [8], we also change the criterion to the TDM ratio. This criterion goes the same when dropping the bad solutions at each refinement level. When choosing the best vertices to move in different refinement methods, a vertex gain function will be defined to evaluate the benefit, originally only considering the cutsizes. Now we consider the cutsizes with the interconnection constraint together.

The pseudocode of our interconnection-aware vertex gain function is shown in Algorithm 1. From lines 5 to 10, we calculate the cutsizes of the vertex move, changing the source and drain nodes if necessary. From lines 12 to 23, we separately calculate the gain on interconnection and topology constraint. We traverse all the FPGA pairs: (a) if they are connected, we calculate the TDM ratio and multiply it by the interconnection violations (lines 20-21); (b) if they are not connected, we estimate the routing cost by multiplying the cutsizes between them by their distance, and check whether the move leads to more topology violations (lines 17-18).

When we get a feasible solution under the topology constraint, no more topology violations should be allowed. So, if the vertex move causes more topology violations (routing distance exceeds the maximum allowed hop), it will have the least priority in the refinement process (lines 24-25). Otherwise, for the interconnection constraint, we set its weight factor to a variable τ that changes with the refinement level. At the beginning, we set τ to 0, and increase it by 1 in the next level. The reason for this is that at higher levels, moves of vertices often cause great changes in cutsizes, and it is hard to refine it back if we place too much emphasis on the interconnection constraint. At the last few levels of the refinement phase, we can mainly focus on reducing the interconnection violations and TDM ratio, because we suppose the initial partitioning phase will not get

Algorithm 1 Interconnection-aware vertex gain function

Input: An initial partition, Vertex v_i , FPGAs $\hat{v}_f$ and $\hat{v}_t$

Output: The gain of moving v_i from $\hat{v}_f$ to $\hat{v}_t$

```

1:  $cut_1 \leftarrow 0, cut_2 \leftarrow 0$  ▷ cutsizes related to  $v_i$ 
   ▷ initialize the cutsizes related to  $v_i$  between FPGAs
2: for each  $(\hat{v}_i, \hat{v}_j) \in \hat{V} \times \hat{V}, i < j$  do
3:    $C_1(\hat{v}_i, \hat{v}_j) \leftarrow 0, C_2(\hat{v}_i, \hat{v}_j) \leftarrow 0,$ 
4: end for
5: for each  $e \in \{e | v_i \in e, e \in E\}$  do
6:    $cut_1 \leftarrow cut_1 + c(e)$ , update relevant elements in  $C_1$ 
7:   update  $S(e)$  and  $D(e)$  for moving  $v_i$  from  $\hat{v}_f$  to  $\hat{v}_t$ 
8:    $cut_2 \leftarrow cut_2 + c(e)$ , update relevant elements in  $C_2$ 
9:   update  $S(e)$  and  $D(e)$  for moving  $v_i$  back to  $\hat{v}_f$ 
10: end for
11:  $gain_{cut} \leftarrow cut_1 - cut_2, gain_{io} \leftarrow 0, gain_{tp} \leftarrow 0$ 
12: for each  $(\hat{v}_i, \hat{v}_j) \in \hat{V} \times \hat{V}, i < j$  do
13:    $\delta \leftarrow C_1(\hat{v}_i, \hat{v}_j) - C_2(\hat{v}_i, \hat{v}_j)$ 
14:    $io_1 \leftarrow \max(C(\hat{v}_i, \hat{v}_j) - IO(\hat{v}_i, \hat{v}_j), 0)$ 
15:    $io_2 \leftarrow \max(C(\hat{v}_i, \hat{v}_j) - IO(\hat{v}_i, \hat{v}_j) - \delta, 0)$ 
16:   if  $\hat{e}(\hat{v}_i, \hat{v}_j) \notin \hat{E}$  then ▷ the same as  $IO(\hat{v}_i, \hat{v}_j) = 0$ 
17:      $gain_{io} \leftarrow gain_{io} + (io_1 - io_2) \cdot dist(\hat{v}_i, \hat{v}_j)$ 
18:     update  $gain_{tp}$  based on  $C_1(\hat{v}_i, \hat{v}_j)$  and  $C_2(\hat{v}_i, \hat{v}_j)$ 
19:   else
20:      $tdm \leftarrow \max(\frac{C(\hat{v}_i, \hat{v}_j)}{IO(\hat{v}_i, \hat{v}_j)}, 1.0)$ 
21:      $gain_{io} \leftarrow gain_{io} + (io_1 - io_2) \cdot tdm$ 
22:   end if
23: end for
24: if  $gain_{tp} > 0$  then ▷ cause more topology violations
25:   return  $-\infty$  ▷ most unbeneficial vertex move
26: else
27:   return  $gain_{cut} + gain_{io} \cdot \tau$ 
28: end if

```

rather bad solutions and only a few moves are needed to get a better result.

During the calculation of the gain function, we need two global variables: the cut matrix C and the interconnection resource matrix IO . The latter will not change throughout the partitioning process, but the former will change constantly after each move. Since recalculating the cut matrix may lead to changes in the gain function of every vertex, we only update the cut matrix after a single round of the PM process to reduce time. For the greedy K-way refinement, we do the update right after each move. We assume that this lazy update will not affect the final result too much, and if it does, the greedy K-way refinement will correct it to a certain extent.

3.3 Min-hop Topology-driven Initial Partitioning

The topology constraint is a special case of interconnection constraint, where the interconnection resources between FPGAs are limited to 0. Therefore, modifications should also be applied to the whole process of partitioning as in the previous subsection to maintain the direction of signal flow in the original netlist. The main difference is that we need to calculate the candidate set $cddt$ beforehand and replace the initial partitioning algorithm with a topology-driven one, which will be discussed as follows.

The initial partitioning algorithms in [17, 24] can find a non-hop solution without topology violations effectively if the solution exists, but have the following drawbacks. Firstly, they only consider the cutsize when choosing FPGAs, but ignore the importance of its vertex degree, i.e. the number of its neighbors. Another problem is that if the algorithm fails at the coarsest level, it will either report no solution[24] or give a solution with minimum violation[17], giving up the opportunity to find better solutions at upper levels. Besides, it is often difficult to find a non-hop solution when the topology of MFS is quite complicated. So, it is necessary to relax the non-hop constraint in the partitioning process. In our approach, we first do the candidate FPGA propagation as [24], and ensure the size of each cluster vertex is no less than a threshold β . Then, we transform the hypergraph into a simple graph, where each hyperedge e is transformed into $|e| - 1$ edges from the source node to the drain nodes, and run the min-hop topology-driven initial partitioning algorithm as Algorithm 2. We try to find a solution with the maximum hop no more than a given threshold $allow_hop$. If we cannot find a solution, we will continue tracing back to the less coarsened graph until we find one. Otherwise, we will search with a larger $allow_hop$.

Algorithm 2 Min-hop topology-driven initial partitioning

Input: Converted simple graph $G'(V', E')$, fixed nodes V'_{fix} , maximum allowed hop $allow_hop$

Output: Partition result $part$

```

1:  $\forall v \in V', part(v) = -1$ 
2: assign fixed nodes in  $V'_{fix}$  to the corresponding FPGA
3: for  $h = 0; h \leq allow\_hop; h++$  do
     $\triangleright Q$ : sort up by  $|cddt(v)|$ , break ties by  $degree(v)$ 
     $\triangleright R$ : sort down by  $degree(\hat{v})$ , break ties by cutsize
4: priority queue  $Q \leftarrow \emptyset, R \leftarrow \emptyset$ 
5:  $\forall$  unassigned  $v, Q.insert(\{v, |cddt(v)|, degree(v)\})$ 
6:  $F \leftarrow \emptyset$   $\triangleright$  failed nodes that has no available FPGA
7: while  $Q \neq \emptyset$  do
8:    $v \leftarrow Q.pop(), R \leftarrow \emptyset$ 
9:   for each  $\hat{v} \in cddt(v)$  do
     $\triangleright$  check resources on  $\hat{v}$  first if needed
10:    calculate cut increment  $\Delta(v)$  if  $part(v) = \hat{v}$ 
11:     $R.insert(\{\hat{v}, degree(\hat{v}), \Delta(v)\})$ 
12:   end for
13:   if  $R = \emptyset$  then  $\triangleright$  no candidates or no resources
14:      $F.push\_back(v)$ 
15:     continue
16:   else
17:      $part(v) \leftarrow R.pop()$   $\triangleright$  find the best FPGA
18:     propagate  $v$  by method in [17] with slack  $h$ 
19:   end if
20: end while
21: if  $F \neq \emptyset$  then
22:    $\forall v \in F$ , recalculate  $cddt(v)$  with slack  $h + 1$ 
23: else
24:   return  $part$   $\triangleright$  get a partition with minimum hop
25: end if
26: end for

```

During each iteration of Algorithm 2, when selecting the order to iterate vertices, we first sort them by the size of $cddt$ in ascending order, and break ties by favoring vertices with higher degrees. When choosing the best FPGA for a node, we first sort them by the number of its neighbors in descending order, and break ties by favoring FPGAs with smaller cutsize increments. After assigning v to $part(v)$, we do the propagation like [17], but with a slack h (lines 16-17), updating $cddt$ of each necessary vertex u using the inequality: $dist(\hat{u}, part(v)) \leq dist(u, v) + h, \forall \hat{u} \in cddt(u)$. When $cddt$ of a node becomes empty due to insufficient resources or topology violation, we mark these nodes as failed nodes F and temporarily skip them (lines 13-14). If failed nodes exist, we recalculate $cddt$ of each failed node with slack $h + 1$ according to the assigned nodes and remaining resources, and continue with the next iteration.

If the non-hop solution cannot be found, a solution with violations will be given. The refinement methods will help find hidden opportunities for feasible solutions, but if the violations still exist after refinement, the inter-FPGA routing is executed to pass signals between FPGA pairs that are not directly connected. That is the main reason for our target of minimizing the hop. For each hyperedge cut that violates the topology of MFS, we start from the source node and find the shortest paths to all the drain nodes. To balance the load on MFS edges, we use an algorithm similar to [25]. We assign every MFS edge a cost, which is positively correlated to the number of nets that go through the MFS edge and is dynamically adjusted during the whole routing process.

4 Experimental Results

In this section, we present the experimental results with the Titan23 benchmark suite[18] and actual industrial designs to validate the proposed techniques. The statistics of these test cases are listed in Table 3, where those of the industrial cases are obtained after the hypergraph modeling phase of EasyPart. Each industry case requires 26 different types of resources. The proposed techniques and EasyPart are compared with hMETIS, TritonPart, and the approach in [17]. The proposed techniques are implemented in C++. All the experiments are carried out on a Linux server with AMD EPYC 7V12 64-Core CPU and 512GB memory.

Table 3: Characteristics of benchmarks.

Benchmark	V	E	Benchmark	V	E
sparcT1_core	91976	92827	minres	261359	320540
neuron	92290	125305	cholesky_bdti	266422	342688
stereo_vision	94050	127085	denoise	275638	356848
des90	111221	139557	sparcT2_core	300109	302663
SLAM_spheric	113115	142408	gsm_switch	493260	507821
cholesky_mc	113250	144948	mes_noc	547544	577664
segmentation	138295	179051	LU230	574372	669477
bitonic_mesh	192064	235328	LU_Network	635456	726999
dart	202354	223301	sparcT1_chip2	820886	821274
openCV	217453	284108	directrf	931275	1374742
stap_qrd	240240	290123	bitcoin_miner	1089284	1448151
industry1	56295	59343	industry3	1102782	1416289
industry2	112575	115615	industry4	1826808	2988069

4.1 Results on the Basic Hypergraph Partitioning

We first validate the effectiveness of the hypergraph partitioning engine in EasyPart. It is compared with two state-of-the-art hypergraph partitioners: hMETIS 2.0pre1 version¹ and TritonPart², on Titan23 benchmarks. Because the two partitioners consider both the upper and lower bounds of the balance constraint, we change the constraint of our engine to align with them. The imbalance factor ϵ is set to 5% and the number of FPGAs K is set to 4. The compared hMETIS supports 64-bit architectures, direct K-way partition, and has more stable performance. It runs with default parameters: nruns = 10, ptype = kway, ctype = h1, rtype = krandom, cmaxnet = 50, rmaxnet = 50, nvcycles = 1, seed = 0. otype = soed is set so that its objective function becomes similar to our setting. For TritonPart, we set the seed = 0 and all other parameter settings as default. CPLEX³ is used as the primary ILP solver to keep the same as [4].

The results of the cutsizes and runtime comparisons are listed in Table 4. For hMETIS and TritonPart, the values in the cutsizes column are recalculated using (4). From the table, we can see that if we set hMETIS as the baseline, TritonPart achieves a cutsizes that is on average 3% smaller, and EasyPart achieves a cutsizes 14% smaller. For most cases, EasyPart achieves the best cutsizes. This validates the effectiveness of our hypergraph partitioning engine which is presented in Section 3.1. Regarding the runtime, EasyPart is 2.69× slower than hMETIS, but still 22× faster than TritonPart. This is partially due to the support of multiple constraints in EasyPart, which makes it run slower than hMETIS. TritonPart runs very slowly in

¹<http://glaros.dtc.umn.edu/gkhome/views/metis>

²We obtain the source code from <https://github.com/ABKGroup/TritonPart>.

³<https://www.ibm.com/analytics/cplex-optimizer>

Table 4: The comparison of hMETIS, TritonPart, and our EasyPart on the Titan23 benchmarks ($K = 4$, $\epsilon = 5\%$).

Benchmark	Cutsizes			Runtime (s)		
	hMETIS	TritonPart	EasyPart	hMETIS	TritonPart	EasyPart
sparcT1_core	3088	3542	2754	11	3941	38
neuron	716	712	529	5	146	24
stereo_vision	518	373	344	6	165	22
des90	883	1027	858	16	148	32
SLAM_spheric	3856	4248	3719	20	7558	40
cholesky_mc	1168	1555	1147	10	592	39
segmentation	589	537	464	16	146	28
bitonic_mesh	1291	1332	1300	31	231	60
dart	1641	1967	1388	19	664	69
openCV	1415	1347	1288	18	225	68
stap_qrd	1150	993	1014	25	190	101
minres	610	543	556	22	260	91
cholesky_bdti	2342	2276	2149	34	4863	98
denoise	1154	1048	987	42	172	52
sparcT2_core	2857	3183	2820	64	235	116
gsm_switch	14201	4483	4065	51	1514	182
mes_noc	1573	1785	1546	87	302	152
LU230	6229	6010	5678	67	10080	161
LU_Network	1754	1073	1062	87	230	140
sparcT1_chip2	2288	2654	1989	128	343	242
directrf	1164	1220	1152	153	253	197
bitcoin_miner	4496	3984	3869	221	1101	365
Avg Ratio	1.00	0.97	0.86	1.00	58.69	2.69

some cases because the ILP solver with large time complexity is involved, especially when the number of vertices is quite large.

4.2 Validating the Proposed Topology-driven Partitioning Approach on Non-hop Setting

To compare the approaches for topology-driven partitioning fairly, we use the fixed nodes and MFS topology[21] the same as TopoPart and [17], for the Titan23 benchmark suite. The original n -pin nets are transformed into $(n - 1)$ two-pin nets and the balance factor is ignored. For the hyperparameter β , TopoPart sets its default value as $\frac{2K}{3}$, while [17] sets it as $\frac{K}{3}$ because it eliminates all the violated status and therefore can have a more strict constraint. Here, we conduct an empirical study on the impact of β on the partitioning results. We set β from $\frac{K}{3}$ to $\frac{2K}{3}$, namely 3, 4 and 5 in this case since $K = 8$.

Because [17] is the enhanced version of TopoPart, we only implement [17] as the baseline with all the default settings. We set the maximum allowed hop of our algorithm to 0, forcing it to get a non-hop solution. Table 5 shows the cutsizes of the partitions produced by [17] and our algorithm across all test cases under different β values. There is no significant difference in the runtime between the two algorithms so we do not list it in the table.

Both algorithms successfully get non-hop solutions for all 22 cases under different β values. The results of [17] when $\beta = 3$ are overall smaller than those in its original paper due to different coarsening and refinement methods. From the results, we can see that our approach achieves approximately the same results as [17] when $\beta = 3$. But when β increases, the cutsizes of our algorithm is relatively steady

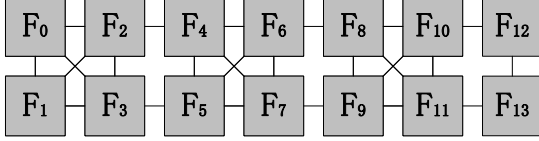
Table 5: The comparison of different initial partitioning methods on the Titan23 benchmarks with non-hop constraint.

Benchmark	$\beta = 3$		$\beta = 4$		$\beta = 5$	
	[17]	Ours	[17]	Ours	[17]	Ours
sparcT1_core	301	301	2954	301	2196	301
neuron	460	451	626	437	1227	415
stereo_vision	164	164	375	163	6620	163
des90	963	963	933	655	655	655
SLAM_spheric	241	525	4405	464	4736	445
cholesky_mc	570	570	1567	426	427	426
segmentation	1156	1156	514	437	404	404
bitonic_mesh	1755	1755	1430	699	881	699
dart	225	225	1494	281	2979	282
openCV	805	843	1592	491	518	516
stap_qrd	474	473	1098	473	4870	473
minres	339	339	548	461	15444	331
cholesky_bdti	796	796	3666	593	1050	540
denoise	818	818	1087	483	1047	430
sparcT2_core	233	233	2968	201	142574	201
gsm_switch	243	243	4534	248	11939	248
mes_noc	814	814	1702	302	317	317
LU230	521	521	8098	321	322	321
LU_Network	1074	450	1120	298	39517	332
sparcT1_chip2	190	190	1808	172	199450	172
directrf	820	820	1561	430	430	430
bitcoin_miner	160	164	3714	164	54369	164
Avg Ratio	1.04	1.00	7.11	1.00	114.06	1.00

Table 6: Results of EasyPart with different working modes for partitioning the industrial cases on the MFS in Fig. 4.

Case	Not consider interconnect/topology constraints					Interconnection-aware					Interconnection-aware + Topology-driven							
	Before routing		After routing			Before routing		After routing			Before routing		After routing					
	Cutsize #Violation		Cutsize	TDM _{max}	TDM _{avg} #Hop	Cutsize #Violation		Cutsize	TDM _{max}	TDM _{avg} #Hop	Cutsize #Violation		Cutsize	TDM _{max}	TDM _{avg} #Hop			
industry1	85	62	186	1.25	1.25	5	103	50	161	1.00	1.00	4	146	30	155	1.00	1.00	2
industry2	148	113	352	2.30	1.68	5	187	71	285	1.45	1.23	5	226	50	266	1.40	1.23	3
industry3	26227	19469	54283	444	272	4	32069	6520	50748	386	231	5	35475	1887	37188	200	200	2
industry4	25572	21856	57438	741	497	5	39734	7535	54324	288	222	4	49206	1847	51413	206	171	3

#Violation is the number of topology violations. TDM_{max} and TDM_{avg} are the maximum and average TDM ratios, respectively. #Hop is the maximum hop.

**Figure 4: Topology of an industrial multi-FPGA system.**

and even achieves significant improvements in many cases, while the cutsize of [17] is quite sensitive to β and has a large fluctuation in some cases, like *sparcT2_core*, *sparcT1_chip2* and *bitcoin_miner*.

The reasons for the above results are as follows. The selection of β is a trade-off between efficiency and quality[24]. Therefore, if β is relatively small, the coarsest hypergraph will have fewer vertices, resulting in less computation but smaller searching space, and often suboptimal solution (or even no solution). In contrast, if β is relatively large, the search space will be larger, along with more computation. For example, *sparcT2_core* is clustered into 134 vertices when $\beta = 3$, but 128255 vertices when $\beta = 5$. Both TopoPart and [17] only consider the size of *cddt* and cutsize during the initial partitioning but ignore the effect of the number of neighbors of FPGA. Aiming for a smaller cutsize may lead to assigning a node to a marginal FPGA, which causes an unintended future increase of cutsize due to the lack of enough tracebacks. This phenomenon more likely happens when the number of vertices is quite large. For a complex MFS topology, it is very hard to find a solution at a high coarsening level. So, returning to lower levels and dealing with a large hypergraph is often unavoidable. All these reveal that our algorithm is more suitable for the non-hop topology-driven partitioning problem.

4.3 Validating EasyPart with Industrial Cases

Interconnection and topology constraints are common in actual MFS. To validate the proposed EasyPart, we carry out experiments on an industrial MFS topology as shown in Fig. 4, with the interconnection between two FPGAs all set to 20. We use EasyPart to partition four actual designs from our industrial partner on this MFS. If completely flattened, these designs include up to 63 million cells. The numbers of cells and nets after the adaptive flattening for the following experiments of partitioning are listed in Table 3.

We first run EasyPart without considering the interconnection/topology constraint. Then, we run it in a mode only considering the interconnection constraint, and finally run it in the mode considering both the two constraints. EasyPart cannot find a non-hop solution on this MFS for the four industrial designs. So, the necessary routing as described in Section 3.3 is added. Before routing, we count the cutsize and topology violations of the partitioning result. After routing, we count the maximum and average TDM ratio, and

the maximum hop of routing paths, which are the main metrics to evaluate the quality of interconnection-aware and topology-driven partitioning. The results are listed in Table 6. Notice that the partitions with smaller TDM ratios and smaller maximum hops have better performances of FPGA-based emulation[23].

From Table 6, we can see that before the routing, although the original partitioning solutions have a relatively small cutsize, they have a large number of topology violations. If the interconnection constraint is considered during the partition, the violations are largely reduced at the cost of some unavoidable cutsize increments. This becomes more obvious when we run EasyPart in full mode which also considers topology constraint at the early stage of partitioning. After the routing, all these topology violations are removed with each running mode of EasyPart. If comparing the three modes, we can see that if both constraints are considered, EasyPart produces the results with the smallest TDM ratios and the smallest maximum hops. This validates the effectiveness of the proposed techniques. We also list the runtimes of EasyPart in full mode in Table 7, including the runtimes for hypergraph modeling and hypergraph partitioning respectively. From it, we can see the efficiency of EasyPart, which costs no more than 23 minutes to deliver a high-quality partition for the industrial case with up to 63 million flattened cells.

Table 7: Runtime breakdown of EasyPart for the industrial cases. #cell is the number of cells after completely flattening.

	industry1	industry2	industry3	industry4
#cell	31,779,127	63,536,575	1,921,478	6,267,124
T _{model} (s)	589	1272	122	480
T _{part} (s)	47	87	511	897

5 Conclusions

In this work, we introduce the constraints-driven circuit netlist hypergraph partitioning problem for FPGA-based logic emulation and present an effective and comprehensive partitioner, EasyPart. Adaptations are made in the multilevel partitioning paradigm to optimize it under this setting with considerations of multiple constraints. We also propose a min-hop initial partitioning algorithm to handle the topology constraint, and a detailed treatment to improve the quality of the partitioning solution under the interconnection constraint. Extensive experimental results show that EasyPart can largely reduce the cutsize compared to the leading partitioners. We also demonstrate the effectiveness of EasyPart in handling the topology and interconnection constraints, achieving better partitioning results with more robustness compared to previous work, and significant improvements in TDM ratio and maximum hop when testing on industrial cases.

References

- [1] Ali Aghdaei and Zhuo Feng. 2022. Hyperef: Spectral hypergraph coarsening by effective-resistance clustering. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*. 1–9.
- [2] Ali Aghdaei, Zhiqiang Zhao, and Zhuo Feng. 2021. Hypersf: Spectral hypergraph coarsening via flow-based local clustering. In *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 1–9.
- [3] Charles J Alpert and Andrew B Kahng. 1995. Recent directions in netlist partitioning: A survey. *Integration* 19, 1-2 (1995), 1–81.
- [4] Ismail Bustany, Grigor Gasparyan, Andrew B Kahng, Ioannis Koutis, Bodhisatta Pramanik, and Zhiang Wang. 2023. An Open-Source Constraints-Driven General Partitioning Multi-Tool for VLSI Physical Design. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–9.
- [5] Ismail Bustany, Andrew B Kahng, Ioannis Koutis, Bodhisatta Pramanik, and Zhiang Wang. 2022. SpecPart: A supervised spectral framework for hypergraph partitioning solution improvement. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*. 1–9.
- [6] Ismail Bustany, Andrew B Kahng, Ioannis Koutis, Bodhisatta Pramanik, and Zhiang Wang. 2023. K-SpecPart: Supervised Embedding Algorithms and Cut Overlay for Improved Hypergraph Partitioning. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2023).
- [7] Ümit V Çatalyürek and Cevdet Aykanat. 2011. PaToH (Partitioning Tool for Hypergraphs).
- [8] Jason Cong and Sung Kyu Lim. 1998. Multiway partitioning with pairwise movement. In *Proceedings of the 1998 IEEE/ACM international conference on Computer-aided design*. 512–516.
- [9] Mehmet Deveci, Kamer Kaya, Bora Uçar, and Ümit V Çatalyürek. 2015. Hypergraph partitioning for multiple communication cost metrics: Model and methods. *J. Parallel and Distrib. Comput.* 77 (2015), 69–83.
- [10] Charles M Fiduccia and Robert M Mattheyses. 1988. A linear-time heuristic for improving network partitions. In *Papers on Twenty-five years of electronic design automation*. 241–247.
- [11] David S Johnson and Michael R Garey. 1979. *Computers and intractability: A guide to the theory of NP-completeness*. WH Freeman.
- [12] George Karypis, Rajat Aggarwal, Vipin Kumar, and Shashi Shekhar. 1999. Multilevel hypergraph partitioning: Applications in VLSI domain. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 7, 1 (1999), 69–79.
- [13] George Karypis and Vipin Kumar. 1998. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on scientific Computing* 20, 1 (1998), 359–392.
- [14] George Karypis and Vipin Kumar. 1999. Multilevel k-way hypergraph partitioning. In *Proceedings of the 36th annual ACM/IEEE design automation conference*. 343–348.
- [15] Thomas Lengauer. 2012. *Combinatorial algorithms for integrated circuit layout*. Springer Science & Business Media.
- [16] Benzheng Li, Zhongdong Qi, Zhengguang Tang, Xiyi He, and Hailong You. 2022. High quality hypergraph partitioning for logic emulation. *Integration* 83 (2022), 67–76.
- [17] Benzheng Li, Hailong You, Shunyang Bi, and Yuming Zhang. 2024. An Efficient Hypergraph Partitioner under Inter-Block Interconnection Constraints. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1–6.
- [18] Kevin E Murray, Scott Whitty, Suyu Liu, Jason Luu, and Vaughn Betz. 2013. Titan: Enabling large and complex benchmarks in academic CAD. In *2013 23rd International Conference on Field programmable Logic and Applications*. IEEE, 1–8.
- [19] Sebastian Schlag, Tobias Heuer, Lars Gottesbüren, Yaroslav Akhremtsev, Christian Schulz, and Peter Sanders. 2023. High-quality hypergraph partitioning. *ACM Journal of Experimental Algorithmics* 27 (2023), 1–39.
- [20] Navaratnasothie Selvakkumaran and George Karypis. 2006. Multiobjective hypergraph-partitioning algorithms for cut and maximum subdomain-degree minimization. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 25, 3 (2006), 504–517.
- [21] Yu-Hsuan Su, Richard Sun, and Pei-Hsin Ho. 2019. 2019 CAD contest: System-level FPGA routing with timing division multiplexing technique. In *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 1–2.
- [22] Aleksandar Trifunovic and William J Knottenbelt. 2004. Par k way 2.0: A Parallel Multilevel Hypergraph Partitioning Tool. In *International Symposium on Computer and Information Sciences*. Springer, 789–800.
- [23] Xinshi Zang, Evangeline FY Young, and Martin DF Wong. 2023. SPARK: A Scalable Partitioning and Routing Framework for Multi-FPGA Systems. In *Proceedings of the Great Lakes Symposium on VLSI 2023*. 593–598.
- [24] Dan Zheng, Xinshi Zang, and Martin DF Wong. 2021. Topopart: a multi-level topology-driven partitioning framework for multi-fpga systems. In *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 1–8.
- [25] Peng Zou, Zhifeng Lin, Xiao Shi, Yingjie Wu, Jianli Chen, Jun Yu, and Yao-Wen Chang. 2020. Time-division multiplexing based system-level FPGA routing for logic verification. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.